4CSPL1012: PROBLEM SOLVING USING PYTHON

Module 1: Algorithmic Problem Solving

Dr. Pandava Sudharshan Babu & Python Faculty Team

School of Engineering and Technology (SoET)

Credits: Respective owners

Rules and announcements:

1. Absolute silence in the class.

2. Discipline to be followed everywhere.

3. Be regular to classes.

4. Mobile phones are strictly not allowed. (Severe action will be taken if caught)

5. Course updates will be posted in the whatsapp group.

Class updates will be shared here

Python_Crash_Course_CSE_A_Y25

WhatsApp group

Scan or upload this QR code using the WhatsApp camera to join this group

Introduction to Algorithms:

- An **algorithm** = step-by-step solution to a problem
- In Python, algorithms are implemented using:

✓ Variables

✓ Control structures

✓ Functions

Example:

```python
a = 5
b = 10
print(a + b)
```

💡 Real-Life Example:

Making Tea 🍵

1. Boil water
2. Add tea powder
3. Add milk
4. Add sugar
5. Serve

👉 If steps are wrong → bad tea 😄

Another Example:

Algorithm: How students submit assignments

1. Receive assignment
2. Ignore for 5 days
3. Panic 1 day before
4. Google everything
5. Submit at last minute 😅

👉 Still an algorithm (but not efficient!)

Properties of Algorithm

Property	Meaning	Example
Input	Takes one or more inputs	List of numbers
Output	Produces at least one result	Minimum value
Finiteness	Must terminate after finite steps	Loop ends after condition
Definiteness	Steps are clear and unambiguous	Well-defined instructions
Effectiveness	Steps are simple and executable	Basic arithmetic operations

Building Blocks of Algorithms:

Core Components:

1. Statements
2. State
3. Control Flow
4. Functions

1. Statements:

- Basic instructions executed by a computer

📌 Examples:

- Assign value → `x = 10`

- Input → Read number

- Output → Print result

📌 Real-life:

"Take 2 cups of water" → a statement

2. State

- State = current values of all variables at a given time
- It changes as the algorithm executes

📌 Example:

x = 5 → state of x is 5

x = x + 2 → state becomes 7

For example:

x = 10
x = x + 5
x = x - 3

Step	Statement	State of x
1	x = 10	10
2	x = x + 5	15
3	x = x - 3	12

📌 Real-life:

Bank balance changing after transactions 💰

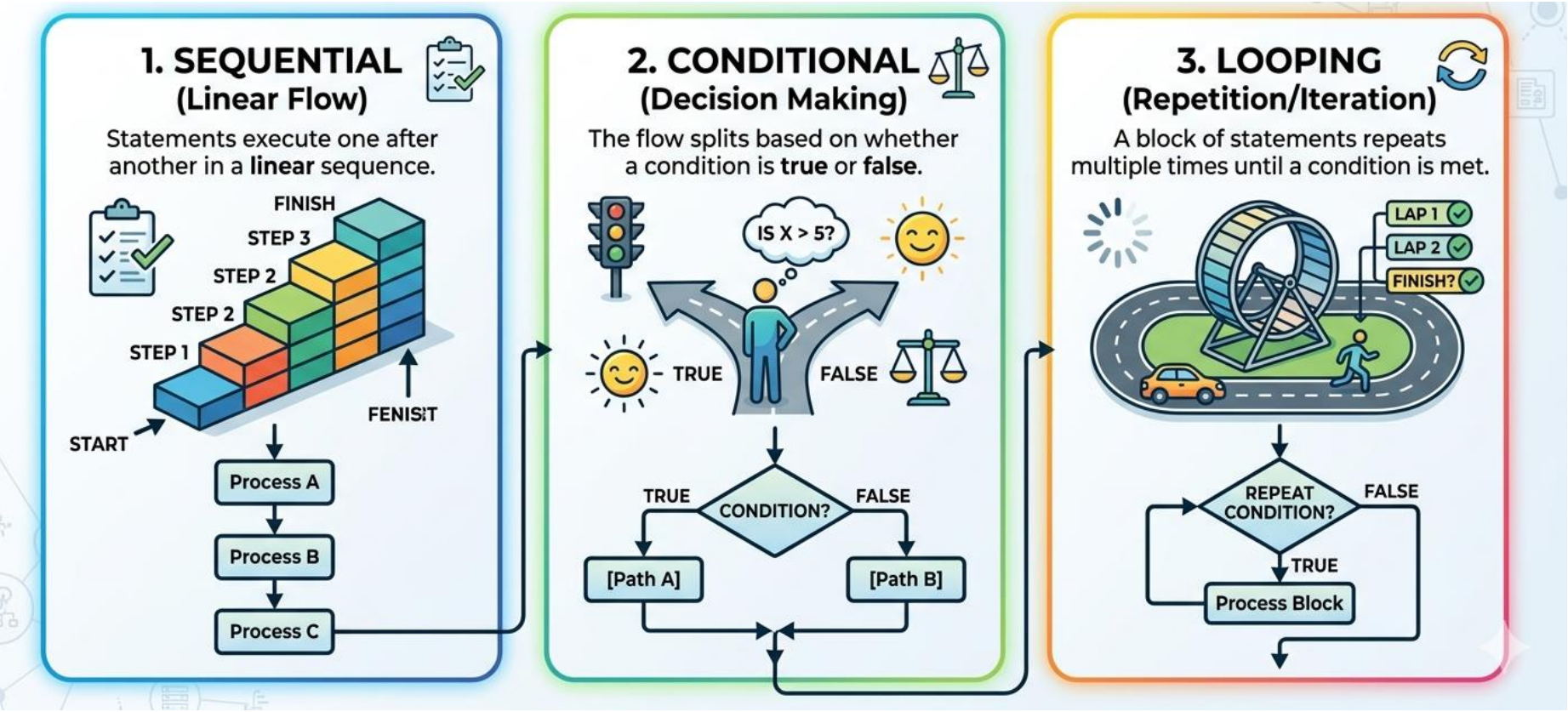
Key Idea: State evolves step-by-step

3. Control Flow:

- Order in which statements execute

Types:

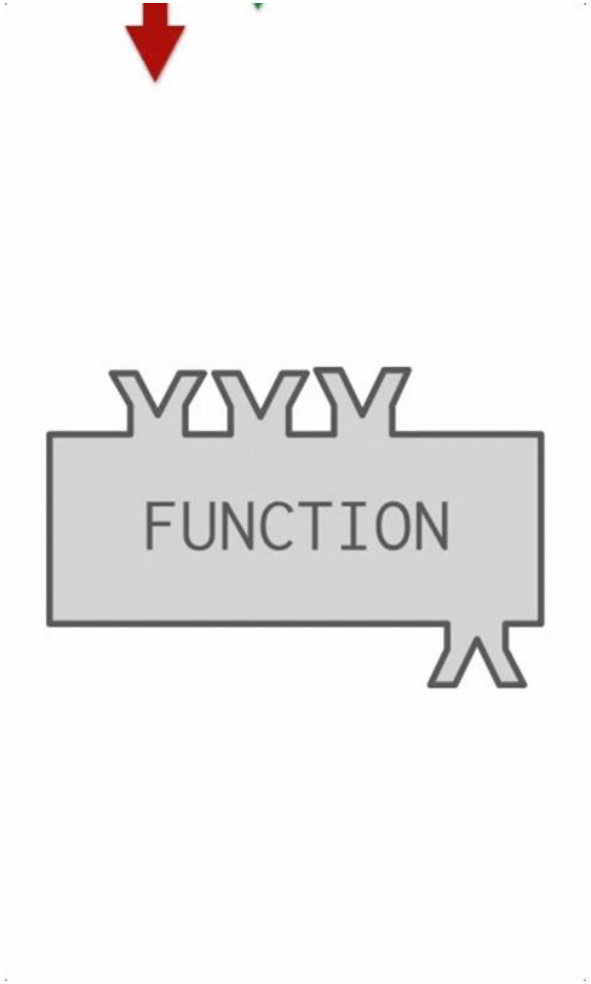
- Sequential
- Conditional
- Looping



Pic: AI Generated

4. Functions:

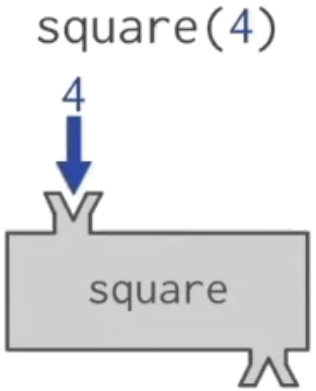
- Reusable blocks of code



📌 Real-life:

ATM withdrawal process 🏧 (same steps reused)

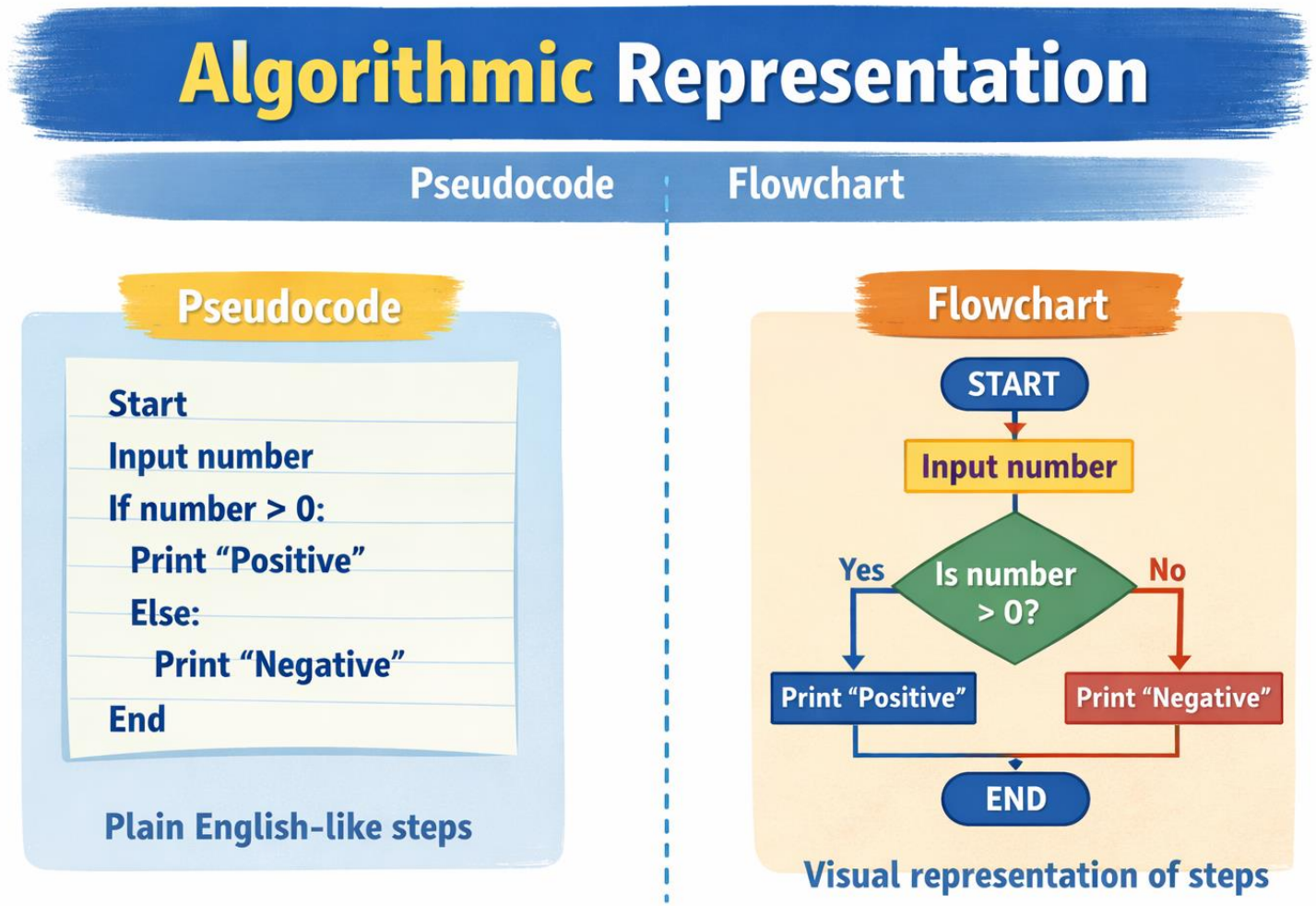
Example:



Algorithm Representation:

Two Main Types:

- Pseudocode
- Flowchart



Control Structures:

- Control structures define the **flow of execution** in an algorithm
- They determine **which statements run and when**

🔑 **Types:**

- Sequence
- Selection (Decision)
- Iteration (Looping)

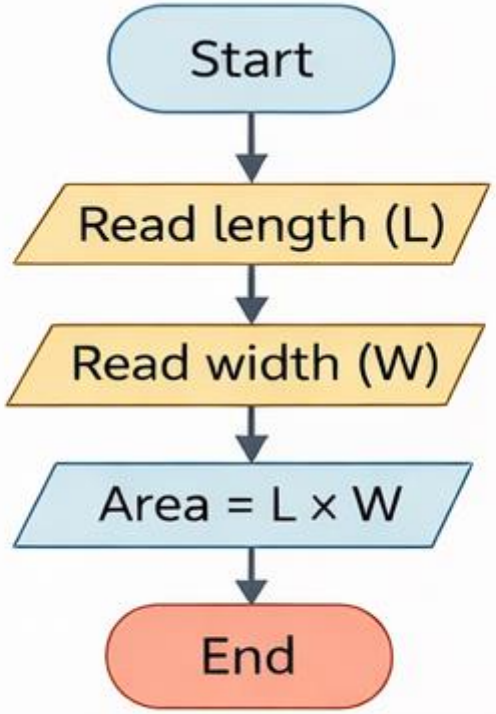
Real-life Analogy:
Daily routine → wake up → brush → eat → go to work

Sequence Control Structure

Algorithm

1. Start
2. Read length (L)
3. Read width (W)
4. Area = L × W
5. Print Area
6. End

Flowchart

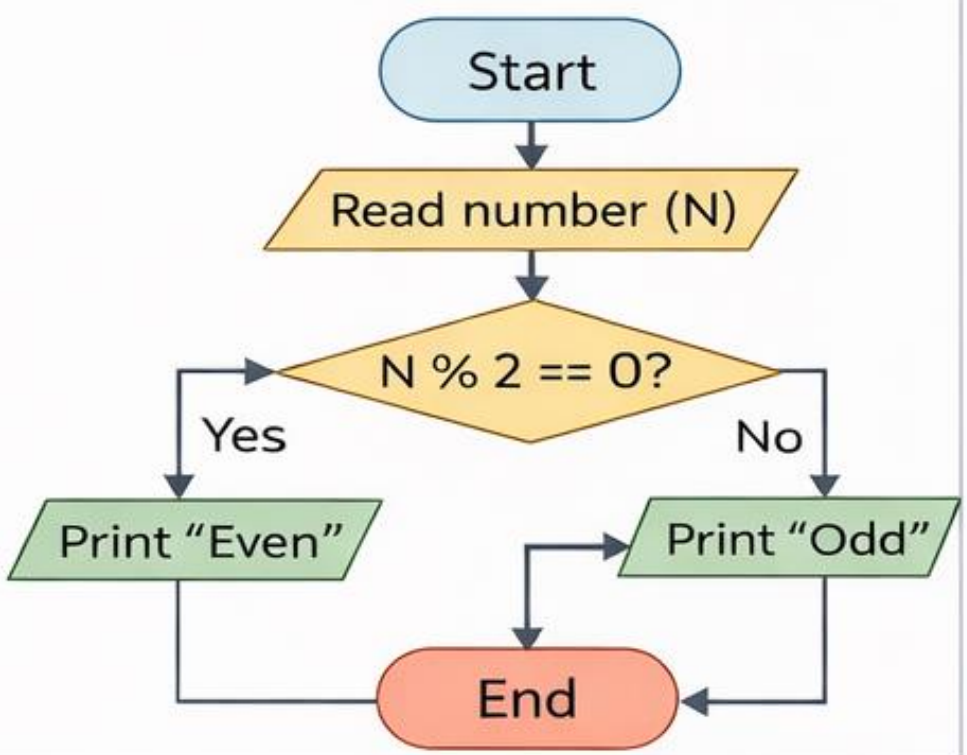


Selection Control Structure (Even/Odd)

Algorithm

1. Start
2. Read number (N)
3. If N % 2 == 0
    4. Print “Even”
    Else
    5. Print “Odd”
6. End

Flowchart

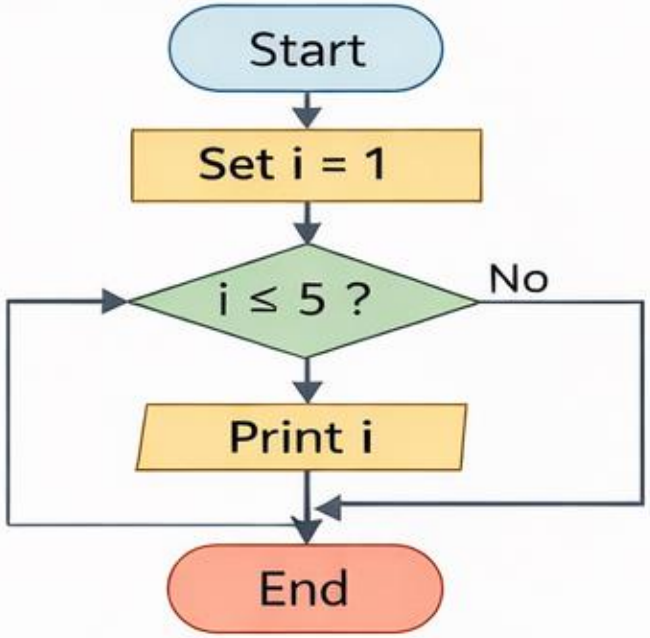


Iteration (Looping) Control Structure (Print 1 to 5)

Algorithm

1. Start
2. Set i = 1
3. While i ≤ 5
    4. Print i
        i = i + + 1
6. End

Flowchart



Introduction to Programming and Python:

- Programming is the process of giving instructions to a computer to perform a task.

Real-life Analogy:

ATM → Insert card → Enter PIN → Withdraw cash

Key Idea:

Computer **does not think**

It only **follows instructions (algorithms)**

What is Python?

- Python is a **high-level, general-purpose programming language** used to develop applications easily.
- "Python is a language that helps humans communicate with computers easily."

Applications of Python:

Domain	Example
AI/ML	Chatbots, Recommendation systems
Web	Websites, APIs
Data Science	Data analysis, visualization
Automation	Scripts, task automation

Real-life Applications of Python:

- Swiggy → backend logic
- Google Maps → shortest path algorithms
-
- Netflix → recommendation system

Features of Python

1. Simple

- Easy to read and write
- Uses English-like syntax

👉 Example explanation:

- Instead of complex symbols, Python looks like normal sentences

2. Interpreted

- Code runs **line-by-line**
- No need to compile fully before execution

"Python executes step by step, not all at once"

3. Platform Independent:

Same code runs on:

Windows
Mac
Linux

Example:

Write once → Run anywhere

Program Execution Flow:

Input → Process → Output

- Input → Data given by user
- Process → Logic/algorithm applied
- Output → Result displayed

Example:

```
Input: Number
Process: Check even/odd
Output: Even / Odd
```

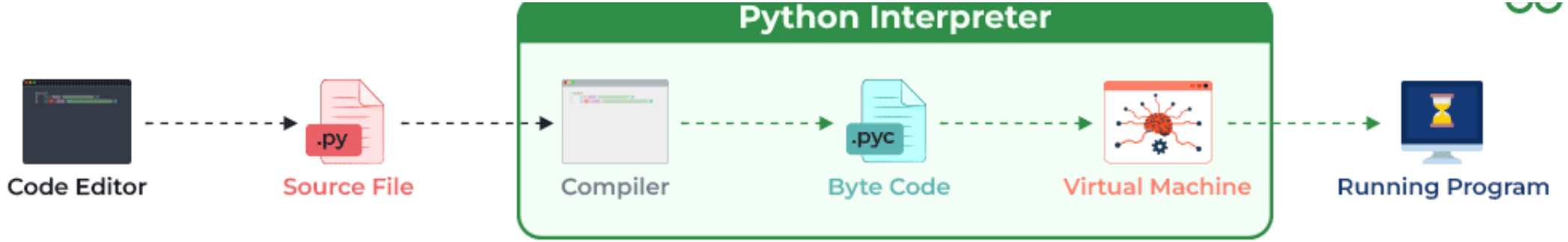
ATM Example

```
Input → Insert card + PIN
Process → Verify PIN + Check balance
Output → Cash / Error message
```

Google Maps

```
Input → Source & destination
Process → Calculate shortest path
Output → Route shown
```

What Happens When Code is Written?



Step	Stage	Description	Output/Result
Step 1	Reading Code	Python reads the source code written in the editor	Code loaded for execution
Step 2	Saving File	Code is saved as a `.py` file containing instructions	Python script file
Step 3	Compilation	Source code is converted into **bytecode** and syntax errors are checked	`.pyc` (bytecode file)
Step 4	Interpretation (PVM)	Python Virtual Machine executes bytecode line by line	Execution begins / errors if any
Step 5	Machine Code Conversion	Bytecode is translated into **machine code (0s and 1s)**	CPU-understandable code
Step 6	Execution	CPU executes instructions and produces result	Final output

Compiler vs Interpreter:

Feature	Compiler	Interpreter
Execution	Whole program	Line-by-line
Speed	Faster after compile	Slower
Errors	After full execution	Immediate
Example	C, C++	Python

Python Execution Flow:

.py (Source Code) → Bytecode (.pyc) → PVM → Machine Code → Output